

Лема за разрастването за контекстносвободни езици.

Незатвореност относно сечение и допълнение

1. Дефиниции

1.1 Контекстносвободна (безконтекстна) граматика

Контекстносвободна граматика (CFG) наричаме наредената четворка

$$G = \langle V, \Sigma, R, S \rangle,$$

където:

- V е крайно множество от **променливи** (нетерминали). Обикновено ги бележим с главни латински букви $A, B, C, S, \dots$;
- Σ е крайно множество от **букви** (терминали), като $V \cap \Sigma = \emptyset$. Обикновено ги бележим с малки латински букви $a, b, c, \dots$;
- $R \subseteq V \times (V \cup \Sigma)^*$ е крайно множество от **правила**. Едно правило $(A, \beta) \in R$ записваме като $A \rightarrow_G \beta$ или просто $A \rightarrow \beta$;
- $S \in V$ е **начална променлива** (стартов нетерминал).

Защо „контекстносвободна“? Лявата страна на всяко правило се състои само от *една променлива* — никакъв контекст. Това е същественото ограничение спрямо контекстните граматики (където лявата страна е $\lambda A \rho$ и заменяме A само в дадения контекст $\lambda \dots \rho$). Тук можем да заместим A с дясната страна на правилото независимо какво я обкръжава.

Защо тази дефиниция е полезна? CFG могат да опишат вложени структури с произволна дълбочина (балансираните скоби, аритметични изрази, синтаксис на езици за програмиране), което крайните автомати не могат — поради което CFL са строго по-богат клас от регулярните езици.

1.2 Извод в CFG

Дефинираме бинарната релация $\Rightarrow_G$ върху $(V \cup \Sigma)^*$ („стъпка на извод“):

$$\frac{(A, \alpha) \in R \quad \lambda, \rho \in (V \cup \Sigma)^*}{\lambda A \rho \Rightarrow_G \lambda \alpha \rho}$$

С думи: вземаме дума, която съдържа променлива A някъде (с контекст λ отляво и ρ отдясно), и заменяме това срещане на A с дясната страна α на правило $A \rightarrow \alpha$.

Релацията $\xRightarrow{\ell}_G$ („извод от ℓ стъпки“) дефинираме индуктивно:

$$\frac{\alpha \in (V \cup \Sigma)^*}{\alpha \xRightarrow{0}_G \alpha} \quad \frac{\alpha \Rightarrow_G \beta \quad \beta \xRightarrow{\ell}_G \gamma}{\alpha \xRightarrow{\ell+1}_G \gamma}$$

И накрая $\alpha \xRightarrow{*}_G \beta$ означава „има *някое* $\ell \in \mathbb{N}$, за което $\alpha \xRightarrow{\ell}_G \beta$ “ — т.е. $\xRightarrow{*}_G$ е *рефлексивното транзитивно затваряне* на $\Rightarrow_G$.

Защо тази многостъпкова релация е важна? Един „извод“ не е една стъпка — той е *последователност* от прилагания на правила, която постепенно превръща стартовата променлива S в дума само над Σ .

1.3 Контекстносвободен език

Език на граматиката G е:

$$\mathcal{L}(G) \stackrel{\text{деф}}{=} \{ \omega \in \Sigma^* \mid S \xRightarrow{*}_G \omega \}.$$

Един език $L \subseteq \Sigma^*$ се нарича **контекстносвободен** (CFL), ако съществува CFG G , такава че $L = \mathcal{L}(G)$.

Прост пример. За $L = \{a^n b^n \mid n \in \mathbb{N}\}$ граматиката $S \rightarrow aSb \mid \varepsilon$ е CFG за L , защото:

- от S можем да получим ε (за $n = 0$);
- от $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow \dots \Rightarrow a^n S b^n \Rightarrow a^n b^n$ — точно по една буква a отляво срещу буква b отдясно, така че броят им винаги съвпада.

1.4 Дърво на извод (синтактично дърво)

Защо ни трябват дървета на извод? Една и съща дума може да се изведе в *различен ред* (можем да преобразуваме лявата променлива първа, после дясната, или обратното) — но интуитивно това са „едни и същи“ изводи. Дървото на извод филтрира този ред: то показва *структурата* на извода, а не реда, в който прилагаме правилата.

Формална дефиниция. Нека $b = \max\{|\gamma| \mid A \rightarrow_G \gamma \text{ е правило}\}$. Двойката $P = (T, \lambda)$ се нарича **дърво на извод съвместимо с G** , ако:

- T е непразно крайно дърво и $T \subseteq \{0, 1, \dots, b-1\}^*$;

- $\lambda : T \rightarrow V \cup \Sigma \cup \{\varepsilon\}$ — *етикетираща функция* — асоциира с всеки възел елемент от граматиката. Пишем $X_\alpha \stackrel{\text{деф}}{=} \lambda(\alpha)$;
- Коренът има етикет от $V \cup \Sigma$, т.е. $\lambda(\varepsilon) \in V \cup \Sigma$;
- Ако $\alpha \in T$ има $k + 1$ преки наследници $\alpha \cdot 0, \alpha \cdot 1, \dots, \alpha \cdot k$, то в граматиката има правило $X_\alpha \rightarrow_G X_{\alpha \cdot 0} X_{\alpha \cdot 1} \cdots X_{\alpha \cdot k}$. Това е *ключовото условие* — то прави дървото „съвместимо“ с граматиката.
- Специален случай: ако $X_\alpha \rightarrow_G \varepsilon$ е правило, позволяваме на α да има единствен наследник $\alpha \cdot 0$ с етикет ε .

Дефинираме:

- $\text{root}(P) = \lambda(\varepsilon)$ — етикетът на корена;
- $\text{yield}(P) = \lambda(\alpha_0)\lambda(\alpha_1) \cdots \lambda(\alpha_k)$, където $\alpha_0, \dots, \alpha_k$ са всички листа подредени **лексикографски** (отляво надясно).

Връзка с езика: $\omega \in \mathcal{L}(G)$ точно тогава, когато съществува дърво на извод P , съвместимо с G , такова че $\text{root}(P) = S$ и $\text{yield}(P) = \omega$.

Пример. За граматиката $S \rightarrow aSb \mid \varepsilon$ и думата $aabb$, дървото е:

```

      S
     / | \
    a  S  b
     / | \
    a  S  b
       |
       ε

```

Тук $\text{yield} = a \cdot a \cdot \varepsilon \cdot b \cdot b = aabb$. Височината на дървото е 3.

1.5 Еднозначни граматикати

Ако всяка дума от $\mathcal{L}(G)$ има точно **едно** дърво на извод, граматиката е **еднозначна** (unambiguous). Това е важно за компилатори: различните дървета съответстват на различен „смисъл“ на думата.

Пример на нееднозначност. $S \rightarrow S + S \mid S * S \mid 0 \mid 1 \mid \cdots \mid 9$ е нееднозначна: думата $5 * 6 + 2$ има две дървета — едно отговарящо на $(5 * 6) + 2$, друго на $5 * (6 + 2)$.

2. Затвореност относно регулярните операции

Регулярните операции са **обединение, конкатенация и звезда на Клини**. Тук формулираме как от CFG за L_1 и L_2 строим CFG за $L_1 \cup L_2$, $L_1 \cdot L_2$, L_1^* .

***Конвенция.** Преди да правим конструкциите, преименуваме променливите на двете граматики, така че $V_1 \cap V_2 = \emptyset$. Това е съществено — иначе променливите им ще се „омесят“ и изводът ще се обърка. Преименуването не променя езика.*

Твърдение (без доказателство). Класът на CFL е затворен относно обединение, конкатенация и звезда на Клини.

2.1 Обединение: $L_1 \cup L_2$

Нека $G_1 = \langle V_1, \Sigma, R_1, S_1 \rangle$ и $G_2 = \langle V_2, \Sigma, R_2, S_2 \rangle$ са CFG за L_1 и L_2 съответно. Строим:

$$G_{\cup} = \langle V_1 \cup V_2 \cup \{S\}, \Sigma, R_1 \cup R_2 \cup \{S \rightarrow S_1, S \rightarrow S_2\}, S \rangle,$$

където S е нова променлива.

Защо работи. От новата начална S имаме две възможни първи стъпки:

- $S \Rightarrow S_1 \xRightarrow{*} \omega$ — извежда се точно произволна дума от L_1 ;
- $S \Rightarrow S_2 \xRightarrow{*} \omega$ — извежда се точно произволна дума от L_2 .

Тъй като $V_1 \cap V_2 = \emptyset$, единият извод не може да „зарази“ другия — щом сме тръгнали по S_1 , използваме само правила от R_1 .

2.2 Конкатенация: $L_1 \cdot L_2$

Строим:

$$G_{\cdot} = \langle V_1 \cup V_2 \cup \{S\}, \Sigma, R_1 \cup R_2 \cup \{S \rightarrow S_1 S_2\}, S \rangle.$$

Защо работи. От S единствената стъпка е $S \Rightarrow S_1 S_2$. След това разклоняваме отделно S_1 и S_2 : $S_1 \xRightarrow{*} \omega_1 \in L_1$ и $S_2 \xRightarrow{*} \omega_2 \in L_2$. Тъй като двете части ползват разделни нетерминали, не си пречат — резултатът е $\omega_1 \omega_2$.

2.3 Звезда на Клини: L_1^*

Строим:

$$G_* = \langle V_1 \cup \{S\}, \Sigma, R_1 \cup \{S \rightarrow SS_1, S \rightarrow \varepsilon\}, S \rangle.$$

Защо работи. Правилата $S \rightarrow SS_1 \mid \varepsilon$ позволяват S да породи произволен брой S_1 -та едно след друго:

$$S \Rightarrow SS_1 \Rightarrow SS_1S_1 \Rightarrow \cdots \Rightarrow \underbrace{S_1S_1 \cdots S_1}_{n \text{ пъти}} \xRightarrow{*} \omega_1\omega_2 \cdots \omega_n,$$

където всяко $\omega_i \in L_1$. Включваме и ε (за $n = 0$) чрез правилото $S \rightarrow \varepsilon$.

2.4 Други затворености — обобщителна забележка

CFL са затворени и относно:

- **обръщане** L^{rev} : за всяко правило $A \rightarrow X_1X_2 \cdots X_n$ в G строим $A \rightarrow X_n \cdots X_2X_1$ в G^{rev} ;
- **сечение с регулярен език** $L \cap R$ (виж раздел 5.3) — особено важно за доказване, че даден език не е CFL;
- **хомоморфизъм и обратен хомоморфизъм**.

Обаче — **CFL не са затворени относно сечение и допълнение** (виж раздел 5).

3. Лема за разрастването за контекстносвободни езици

3.1 Интуиция

В една CFG броят на променливите $|V|$ е *фиксиран*. Ако една дума ω е достатъчно дълга, нейното дърво на извод трябва да е достатъчно високо. От принципа на Дирихле, по някакъв път от корена до листо ще се срещне *повтаряща се променлива* A . Това повторение позволява да „напомпваме“ частта от дървото между двете срещания на A — да повтаряме съответната подструктура произволно много пъти, или да я изтрием. Така получаваме нови думи, които също трябва да са в езика.

3.2 Формулировка

Лема (за разрастването/покачването за CFL). За всеки безконтекстен език L съществува константа $p \geq 1$ („константа на покачването“), такава че за всяка дума $\alpha \in L$, за която $|\alpha| \geq p$, съществува разбиване на α на пет части

$$\alpha = xyuvw,$$

за което са изпълнени:

1. $|yv| \geq 1$ — частите y и v не са едновременно празни;
2. $|yuv| \leq p$ — средната зона е къса;
3. $(\forall i \in \mathbb{N}) [xy^iuv^iw \in L]$ — можем да повтаряме y и v синхронно колкото пъти искаме (включително $i = 0$).

3.3 Спомагателни твърдения

Преди доказателството ни трябва три лесни наблюдения.

Твърдение А (свързване на изводи). Ако $A \xRightarrow{\ell_1} \alpha_1 B \alpha_3$ и $B \xRightarrow{\ell_2} \alpha_2$, то $A \xRightarrow{\leq \ell_1 + \ell_2} \alpha_1 \alpha_2 \alpha_3$.

Защо? Свързваме двете дървета: в дървото на първия извод сменяме листото с етикет B с цялото дърво на втория извод. Така получаваме едно общо дърво.

Твърдение Б (итериране). Ако $A \xRightarrow{\ell} \beta A \gamma$, то $A \xRightarrow{\leq i \cdot \ell} \beta^i A \gamma^i$ за всяко $i \in \mathbb{N}$.

Защо? Прилагаме многократно същия извод: всяко срещане на A замества се със собственото си дърво. Получаваме i -кратно вложена структура $\beta \beta \cdots A \cdots \gamma \gamma$.

Твърдение В (връзка дължина–височина). Нека $b = \max\{|\gamma| \mid A \rightarrow \gamma \in R\}$ е максималната дължина на дясна страна на правило. Ако $A \xRightarrow{\ell} \alpha$, то $|\alpha| \leq b^\ell$.

Защо? В дървото на извод всеки вътрешен възел има най-много b преки наследници. Дървото има височина $\leq \ell$, следователно листата (които образуват α) са най-много b^ℓ .

Следствие. Ако $\alpha \in \mathcal{L}(G)$ и $|\alpha| > b^\ell$, то всяко дърво на извод за α има височина $\geq \ell + 1$.

3.4 Доказателство на лемата

Избор на константата. Нека $G = \langle V, \Sigma, R, S \rangle$ е CFG за L . Дефинираме:

$$p \stackrel{\text{деф}}{=} b^{|V|+1}, \quad \text{където } b = \max\{|\gamma| \mid A \rightarrow \gamma \in R\}.$$

Защо точно $|V| + 1$? Защото искаме по всеки максимален път от корена до листо да са се срещнали поне $|V| + 1$ променливи — тогава по принципа на Дирихле задължително има повторение (има само $|V|$ различни променливи).

Стъпка 1: избор на дървото. Нека $\alpha \in L$, $|\alpha| \geq p$. От Следствието получаваме, че всяко дърво на извод за α от S има височина $\geq |V| + 1$.

Сред всички такива дървета избираме *най-малкото* $P = (T, \lambda)$ — с минимален брой възли. Това ще е важно за стъпка 4.

Стъпка 2: намиране на повторение. Във P фиксираме максимален път π — от корена до листо. По дължината му се срещат поне $|V| + 1$ променливи. По принципа на Дирихле (с $|V|$ кутии), някоя променлива A се повтаря.

От всички такива повторения избираме това, което е *най-близо до листото*. Това разделя дървото на три зони:

```

      S
     /\
    x  w    <-- (1) Извод (1): S =*> x A w
     |
     A    <-- горно срещане на A
    /\
   y . v    <-- (2) Извод (2): A =≤|V|+1=> y A v
     |
     A    <-- долно срещане на A (избрано най-близо до листо)
     |
     u    <-- (3) Извод (3): A =≤|V|=> u (тук вече няма повторение)
  
```

Дървото се разцепва на три отделни поддървета:

- **(1)** с корен S и изход xAw , дава извод $S \stackrel{*}{\Rightarrow} xAw$;
- **(2)** с корен A (горно срещане) и изход yAv , дава извод $A \stackrel{\leq|V|+1}{\Rightarrow} yAv$. Височината е $\leq |V| + 1$, защото от горното до долното срещане пътят минава през най-много $|V| + 1$ нива (имаме точно $|V| + 1$ различни променливи по пътя си преди да се случи повторение);
- **(3)** с корен A (долно срещане) и изход u , дава извод $A \stackrel{\leq|V|}{\Rightarrow} u$.

Получаваме разбиване $\alpha = xyuvw$.

Стъпка 3: проверка на условие 2: $|yuv| \leq p$.

Изводът (2)+(3), свързан чрез Твърдение А, дава $A \stackrel{\leq |V|+1}{\Rightarrow} yuv$ (прикачваме (3) на мястото на долното A в (2)). По Твърдение В:

$$|yuv| \leq b^{|V|+1} = p. \quad \checkmark$$

Стъпка 4: проверка на условие 1: $|yv| \geq 1$.

Доказваме с противоречие. Допускаме $|yv| = 0$, т.е. и y , и v са празни думи. Тогава:

$$\alpha = xyuvw = xuw,$$

и имаме изводите $S \stackrel{*}{\Rightarrow} xAw$ (от дърво (1)) и $A \stackrel{*}{\Rightarrow} u$ (от дърво (3)) — пропускаме *изцяло* дърво (2)!). Чрез Твърдение А, можем да съединим тези два извода в едно дърво, което извежда $xuw = \alpha$ от S . Но **това ново дърво има по-малко възли от P** — изтрили сме поне един възел (възела с горното A , плюс цялото му под-дърво до долното A).

Това противоречи на избора на P като *най-малкото* дърво. Следователно $|yv| \geq 1$.

✓

Защо изборът на минимално дърво е същественият трик? Точно той гарантира, че y и v не могат и двете да са празни. Ако позволим произволно дърво, лемата би била тривиална — но безсилна, защото бихме могли да „пумпаме нищо“ и не бихме имали контрол.

Стъпка 5: проверка на условие 3: $\forall i \in \mathbb{N}, xy^i uv^i w \in L$.

Сглобяваме извода стъпка по стъпка:

1. От Твърдение Б, изводът $A \stackrel{*}{\Rightarrow} yAv$ дава $A \stackrel{*}{\Rightarrow} y^i Av^i$ за всяко $i \geq 0$.
2. От Твърдение А (свързване с дърво (3)) — $A \stackrel{*}{\Rightarrow} u$: $A \stackrel{*}{\Rightarrow} y^i uv^i$.
3. Отново Твърдение А (свързване с дърво (1)) — $S \stackrel{*}{\Rightarrow} xAw$: $S \stackrel{*}{\Rightarrow} x \cdot y^i uv^i \cdot w$.

Следователно $xy^i uv^i w \in \mathcal{L}(G) = L$ за всяко $i \in \mathbb{N}$. ✓

3.5 Контрапозиция — критерий, че език НЕ е CFL

Лемата казва: „ако L е CFL, то има p с горните свойства“. Контрапозицията дава следния практически критерий:

Следствие. Нека L е безкраен език. Ако:

- за всяко $p \geq 1$,

- съществува дума $\alpha \in L$, $|\alpha| \geq p$, такава че
- за всяко разбиване $\alpha = xyuvw$ с $|yv| \geq 1$ и $|yuv| \leq p$,
- съществува $i \in \mathbb{N}$, за което $xy^iuv^iw \notin L$,

то L **не е** контекстносвободен.

Логическа структура: $\forall \exists \forall \exists$. Това е игра между нас и противник:

1. Противникът ни дава произволно p .
2. Ние избираме конкретна неудобна дума α .
3. Противникът избира произволно разбиване със свойствата.
4. Ние посочваме i , при което xy^iuv^iw изпада от езика.

Печелим, ако имаме стратегия за стъпки 2 и 4 срещу всеки ход на противника в стъпки 1 и 3.

4. Пример: неконтекстносвободен език

Твърдение. $L = \{a^n b^n c^n \mid n \in \mathbb{N}\}$ не е CFL.

Доказателство (по контрапозицията на лемата).

- $(\forall p)$ Нека $p \geq 1$ е произволна константа.
- $(\exists \alpha)$ Избираме $\alpha = a^p b^p c^p$. Очевидно $\alpha \in L$ и $|\alpha| = 3p \geq p$.
- $(\forall \text{ разбиване})$ Нека $\alpha = xyuvw$ е произволно разбиване с $|yv| \geq 1$ и $|yuv| \leq p$.
- $(\exists i)$ Ще покажем, че $i = 2$ работи: $xy^2uv^2w \notin L$.

Ключово наблюдение: Тъй като $|yuv| \leq p$, „прозорецът“ yuv е по-къс от p . Думата $a^p b^p c^p$ има три блока, всеки от дължина p . Прозорецът yuv **не може** да обхване и трите букви a, b, c едновременно — за това би трябвало да премине през два цели блока ($2p$ символа), което е $> p$.

Анализираме случаите:

Случай 1: y и v са съставени от **по една буква** (всяка от тях е изцяло в един от трите блока).

Тогава прозорецът uvw обхваща най-много 2 от 3-те типа букви — поне един тип остава непокътнат. След помпване с $i = 2$, броят на поне един тип букви нараства, но броят на „непокътнатия“ тип остава p . Резултат: бройките на a , b и c вече не съвпадат, така че $xy^2uv^2w \notin L$.

Случай 2: u или v съдържа две различни букви (пресича границата между два съседни блока, например $u = a^jb^k$).

Тогава xy^2uv^2w съдържа подниз от вид $\dots a^jb^ka^jb^k \dots$ — но всички думи в L имат вид $a^*b^*c^*$ с буквите в този ред. Получената дума нарушава реда — следователно $xy^2uv^2w \notin L$.

Случай 3: uvw обхваща и трите блока. Невъзможно, защото това би изисквало $|uvw| > p$.

Във всички възможни случаи $i = 2$ работи: $xy^2uv^2w \notin L$.

От контрапозицията на лемата за разрастването, $L = \{a^n b^n c^n\}$ **не е** контекстносвободен. ■

4.1 Какво научаваме от този пример

Изборът на „пумпваща“ дума α е **половината от изкуството** при решаването на такива задачи. Принципи:

- Думата трябва да е **достатъчно дълга** — обикновено всички степени се избират равни на p или кратни на p ;
- Думата трябва да е **достатъчно стегната** — да не дава на противника удобни разбивания;
- Често помага да се избере думата така, че **прозорецът uvw от p символа да не може да обхване „критичните“ части едновременно**.

5. Незатвореност относно сечение и допълнение

5.1 CFL не са затворени относно сечение

Твърдение. Съществуват CFL L_1, L_2 , такива че $L_1 \cap L_2$ **не е** CFL.

Доказателство (с конкретен контрапример). Разглеждаме езиците:

$$L_1 = \{a^n b^n c^m \mid n, m \in \mathbb{N}\}, \quad L_2 = \{a^m b^n c^n \mid n, m \in \mathbb{N}\}.$$

Защо L_1 е CFL? Защото можем да го опишем като конкатенация: $L_1 = \{a^n b^n \mid n \in \mathbb{N}\} \cdot \{c^m \mid m \in \mathbb{N}\}$. Първият фактор е CFL ($S \rightarrow aSb \mid \varepsilon$), вторият е регулярен (а значи и CFL). Понеже CFL са затворени относно конкатенация, L_1 е CFL. Конкретна граматика:

$$S \rightarrow AB, \quad A \rightarrow aAb \mid \varepsilon, \quad B \rightarrow cB \mid \varepsilon.$$

Защо L_2 е CFL? Аналогично: $L_2 = \{a^m\} \cdot \{b^n c^n\}$ с граматика

$$S \rightarrow AB, \quad A \rightarrow aA \mid \varepsilon, \quad B \rightarrow bBc \mid \varepsilon.$$

Какво е пресечението? Една дума е в $L_1 \cap L_2$ т.с.т.к. има вид $a^* b^* c^*$ и едновременно:

- от L_1 : броят на a е равен на броя на b ,
- от L_2 : броят на b е равен на броя на c .

Това означава $n_a = n_b = n_c$, т.е.:

$$L_1 \cap L_2 = \{a^n b^n c^n \mid n \in \mathbb{N}\}.$$

Но от Раздел 4 знаем, че този език **не е CFL**.

Следователно класът на CFL не е затворен относно сечение. ■

5.2 CFL не са затворени относно допълнение

Твърдение. Съществува CFL L , такъв че $\overline{L} = \Sigma^* \setminus L$ **не е CFL**.

Доказателство (косвено, чрез законите на Де Морган).

Идея: Класът на CFL **е** затворен относно обединение (виж 2.1). Ако беше затворен и относно допълнение, тогава от Де Морган:

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

би следвало, че CFL са затворени *и* относно сечение — а току-що видяхме, че не са.

Формално: Допускаме противното, че CFL са затворени относно допълнение. Нека L_1, L_2 са CFL от Раздел 5.1.

- $\overline{L_1}, \overline{L_2}$ са CFL (по нашето допускане);
- $\overline{L_1} \cup \overline{L_2}$ е CFL (затвореност относно обединение, доказана);
- $\overline{\overline{L_1} \cup \overline{L_2}}$ е CFL (отново по нашето допускане за допълнение).

Но последното, по Де Морган, е именно $L_1 \cap L_2 = \{a^n b^n c^n\}$, който **не е** CFL.

Противоречие. Следователно CFL не са затворени относно допълнение. ■

Алтернативно директно доказателство. Може да се покаже, че $\{a, b\}^* \setminus \{\omega\omega \mid \omega \in \{a, b\}^*\}$ е CFL, докато $\{\omega\omega \mid \omega \in \{a, b\}^*\}$ не е CFL. Така допълнението на конкретен CFL не е CFL.

5.3 Полезно следствие: $\text{CFL} \cap \text{Regular} = \text{CFL}$

Въпреки незатвореността относно сечение, имаме хубаво свойство:

Теорема. Ако L е CFL и R е регулярен език, то $L \cap R$ е CFL.

Защо? Конструкцията използва *декартово произведение* на стеков автомат за L и краен автомат за R . Симулираме двата автомата едновременно и приемаме само когато и двата приемат.

Това е **изключително полезен инструмент** за доказване, че даден език не е CFL: ако пресечем L с подходящ регулярен език и получим нещо, за което вече знаем, че не е CFL, тогава и оригиналният L не е CFL (по контрапозиция).

Пример на приложение. Нека $L = \{\omega \in \{a, b, c\}^* \mid |\omega|_a = |\omega|_b = |\omega|_c\}$. Дали L е CFL?

Допускаме, че да. Тогава $L \cap \mathcal{L}(a^* b^* c^*)$ също би бил CFL. Но пресечението е именно $\{a^n b^n c^n \mid n \in \mathbb{N}\}$ — не е CFL. Противоречие. Следователно L не е CFL.

6. Примерни задачи

Задача 6.1. Построяване на CFG за даден CFL

Условие. Постройте контекстносвободна граматика за езика

$$L = \{a^i b^j c^k \mid i, j, k \in \mathbb{N}, i = j + k\}.$$

Анализ на структурата

Думите имат вид $a^{j+k} b^j c^k$ — например за $j = 2, k = 1$: $aaa bb c$. Идеята е, че всяко a „балансира“ или с едно b , или с едно c . Имаме j a -та, които ще се балансират с j -те b -та, и k a -та, които ще се балансират с k -те c -та.

Геометрично, структурата изглежда така — две вложени двойки:

a a a	b b c	<- (i=3, j=2, k=1)
┌┐┐	┌┐	<- 2 а-та матват 2 б-та (вътрешна двойка)
└┬┘		<- 1 а матва 1 с (външна двойка)

Външните a -та трябва да са „увити“ около вътрешните, защото c -тата идват *след* b -тата.

Граматика

$$S \rightarrow aSc \mid T, \quad T \rightarrow aTb \mid \varepsilon$$

С думи:

- S управлява *външното* балансиране на a с c . Всяко прилагане на $S \rightarrow aSc$ добавя по едно a най-вляво и едно c най-вдясно. След k прилагания: $S \xRightarrow{k} a^k S c^k$.
- В някой момент превключваме чрез $S \rightarrow T$ към вътрешния слой.
- T управлява *вътрешното* балансиране на a с b . Всяко прилагане на $T \rightarrow aTb$ добавя по едно a най-вляво и едно b най-вдясно (отвътре). След j прилагания: $T \xRightarrow{j} a^j T b^j$.
- Накрая $T \rightarrow \varepsilon$ затваря извода.

Проверка с примери

Извод за $aaabbc$ ($i = 3, j = 2, k = 1$):

$$S \Rightarrow aSc \Rightarrow aTc \Rightarrow a(aTb)c = aaTbc \Rightarrow aa(aTb)bc = aaaTbbc \Rightarrow aaa\varepsilon bbc = aaabbc. \checkmark$$

Извод за $aabb$ ($i = 2, j = 2, k = 0$):

$$S \Rightarrow T \Rightarrow aTb \Rightarrow aaTbb \Rightarrow aabb. \checkmark$$

Извод за $aacc$ ($i = 2, j = 0, k = 2$):

$$S \Rightarrow aSc \Rightarrow aaScc \Rightarrow aaTcc \Rightarrow aa\varepsilon cc = aacc. \checkmark$$

Извод за ε ($i = j = k = 0$):

$$S \Rightarrow T \Rightarrow \varepsilon. \checkmark$$

Обосновка на коректността

Трябва да докажем две посоки: $\mathcal{L}(G) \subseteq L$ и $L \subseteq \mathcal{L}(G)$.

Първо ще докажем две помощни твърдения:

(i) Ако $T \xRightarrow{\ell} \omega \in \Sigma^*$, то $\omega = a^j b^j$ за някое $j \geq 0$.

Доказателство с индукция по ℓ :

- За $\ell = 1$: единственото правило, което прави T да изведе дума над Σ за една стъпка, е $T \rightarrow \varepsilon$. Тогава $\omega = \varepsilon = a^0 b^0$. ✓
- Стъпка: ако $T \xRightarrow{\ell+1} \omega$ за $\ell + 1 \geq 2$, първата стъпка трябва да е $T \Rightarrow aTb$ (защото $T \rightarrow \varepsilon$ завършва за 1 стъпка). Тогава $aTb \xRightarrow{\ell} \omega$, т.е. $\omega = a\omega'b$, където $T \xRightarrow{\ell} \omega'$. По индукционно предположение, $\omega' = a^{j'} b^{j'}$, така че $\omega = a \cdot a^{j'} b^{j'} \cdot b = a^{j'+1} b^{j'+1}$. ✓

(ii) Ако $S \xRightarrow{\ell} \omega \in \Sigma^*$, то $\omega = a^{j+k} b^j c^k$ за някои $j, k \geq 0$.

Доказателство с индукция по ℓ : Първата стъпка от извода е или $S \rightarrow T$, или $S \rightarrow aSc$.

- Ако $S \rightarrow T$: тогава $T \xRightarrow{\ell-1} \omega$, така че по (i), $\omega = a^j b^j$ — взимаме $k = 0$. ✓
- Ако $S \rightarrow aSc$: тогава $aSc \xRightarrow{\ell-1} \omega$, т.е. $\omega = a\omega'c$, където $S \xRightarrow{\ell'} \omega'$ за някое $\ell' < \ell$. По И.П., $\omega' = a^{j+k'} b^j c^{k'}$. Тогава $\omega = a \cdot a^{j+k'} b^j c^{k'} \cdot c = a^{j+(k'+1)} b^j c^{k'+1}$ — взимаме $k = k' + 1$. ✓

Сега доказваме включванията:

$\mathcal{L}(G) \subseteq L$: Ако $\omega \in \mathcal{L}(G)$, тогава $S \xRightarrow{*} \omega$. По (ii), $\omega = a^{j+k} b^j c^k$ за някои $j, k \geq 0$.

Полагаме $i = j + k$ и виждаме, че $\omega \in L$.

$L \subseteq \mathcal{L}(G)$: Нека $\omega = a^{j+k} b^j c^k \in L$. Изводим я:

$$S \xRightarrow{k} a^k S c^k \Rightarrow a^k T c^k \xRightarrow{j} a^k \cdot a^j T b^j \cdot c^k \Rightarrow a^{j+k} b^j c^k.$$

Това завършва доказателството. ■

Задача 6.2. Дали даден език е CFL — с обосновка

Условие. За всеки от следните езици определете дали е CFL и обосновайте:

а) $L_a = \{a^n b^m c^n \mid n, m \in \mathbb{N}\}$; б) $L_b = \{a^n b^n c^n d^n \mid n \in \mathbb{N}\}$; в) $L_c = \{\omega\omega \mid \omega \in \{a, b\}^*\}$.

Решение на (а): $L_a = \{a^n b^m c^n\}$ — да, CFL

Идея. Броят на a -тата трябва да съвпада с броя на c -тата (както вложена скоба), докато b -тата в средата са независими.

Граматика:

$$S \rightarrow aSc \mid B, \quad B \rightarrow bB \mid \varepsilon.$$

Обосновка на коректността.

- $S \rightarrow aSc$ поддържа баланса между броя на a и на c — добавя по едно от двете отвън;
- $S \rightarrow B$ преминаване към „вътрешната“ част (последователност от b -та);
- $B \rightarrow bB \mid \varepsilon$ произволно много b -та (включително 0).

Пример (извод за $a^2b^3c^2 = aabbbcc$):

$$S \Rightarrow aSc \Rightarrow aaScc \Rightarrow aaBcc \xrightarrow{3} aabbbBcc \Rightarrow aabbb\varepsilon cc = aabbbcc. \checkmark$$

Формално, с индукция доказваме:

$$(i) B \xrightarrow{*} \omega \Rightarrow \omega = b^m \text{ за някое } m \geq 0; (ii) S \xrightarrow{*} \omega \Rightarrow \omega = a^n b^m c^n \text{ за някои } n, m \geq 0,$$

и обратното — всяка дума $a^n b^m c^n$ е изводима. Аналогично на Задача 6.1.

Решение на (6): $L_b = \{a^n b^n c^n d^n\}$ — не, не е CFL

Идея. Подобно на $\{a^n b^n c^n\}$, но с четири блока. Един стеков автомат може да „брои“ две съответствия едновременно (като ползва стека), но не и *три или повече*. Доказваме формално чрез лемата за разрастването.

Доказателство.

- $(\forall p)$ Нека $p \geq 1$.
- $(\exists \alpha)$ Избираме $\alpha = a^p b^p c^p d^p \in L_b$. Имаме $|\alpha| = 4p \geq p$.
- $(\forall \text{ разбиване}) \alpha = xyuvw, |yv| \geq 1, |yuv| \leq p$.
- $(\exists i)$ Полагаме $i = 2$.

Поради $|yuv| \leq p$, прозорецът yuv обхваща *най-много два* съседни блока (между всеки два съседни блока има разстояние p символа, така че по-къс от p прозорец не може да обхване три различни блока). Случаи:

Случай 1: yuv е изцяло в един блок. Тогава xy^2uv^2w има само един от четирите блока увеличен. Останалите три блока си остават p , така че бройките вече не съвпадат — извън езика.

Случай 2: uiv пресича границата между два съседни блока (напр. между b и c).

Тогава помпването засяга най-много два блока. Останалите два блока (например a и d) си остават p . Поне един от засегнатите блокове нараства (защото $|yv| \geq 1$). Значи бройките на четирите блока не са вече равни — извън езика.

Случай 3: y или v съдържа букви от два блока. Тогава y^2 или v^2 ще съдържа подниз тип $bcbs$ или $abab$, нарушавайки реда $a < b < c < d$ — извън езика.

Във всички случаи $xy^2uv^2w \notin L_b$. От контрапозицията на лемата, L_b не е CFL. ■

Решение на (в): $L_c = \{\omega\omega \mid \omega \in \{a, b\}^*\}$ — не, не е CFL

Защо този език е „по-труден“ от $\{\omega\omega^{\text{rev}}\}$? Стектът на стековия автомат работи по принципа „последен влязъл — пръв излязъл“ (LIFO). Затова палиндромите $\{\omega\omega^{\text{rev}}\}$ са лесни: натрупваме първата половина в стека, после я сверяваме обратно. Обаче при $\omega\omega$ трябва да сверим двете половини в същия ред — стекът ги връща в обратен ред. Затова интуитивно трябва да очакваме, че L_c не е CFL.

Доказателство.

- $(\forall p)$ Нека $p \geq 1$.
- $(\exists \alpha)$ Избираме $\alpha = a^p b^p a^p b^p$. Тогава $\alpha = \omega\omega$ за $\omega = a^p b^p$, така че $\alpha \in L_c$ и $|\alpha| = 4p \geq p$.

***Защо точно тази дума?** Простата дума a^{2p} не работи: ако $\alpha = a^{2p}$, противникът ще избере $y = a^k$ или $v = a^k$ — но за подходящо i дължината пак ще е четна и буквата ще е само a , така че думата си остава в L_c . Думата $a^p b^p a^p b^p$ има по-сложна структура и не позволява такива евтини помпвания.*

- $(\forall \text{ разбиване}) \alpha = xyuvw, |yv| \geq 1, |yuv| \leq p$.
- $(\exists i)$ Полагаме $i = 2$.

Думата α има четири блока, всеки от дължина p . Прозорецът uiv има дължина $\leq p$, така че не може да обхване и трите средни блокчета — а най-важно, не може да съдържа едновременно символи от първата половина (позиции 1 до $2p$) и втората половина (позиции $2p + 1$ до $4p$), освен ако не е точно около средата (позициите около $2p$).

Главен аргумент за противоречие: показваме, че помпването нарушава структурата „две равни половини“.

Случай 1: uiv е изцяло в първата половина ($a^p b^p$ — позиции 1 до $2p$).

След помпване с $i = 2$, новата дума има вид $X \cdot a^p b^p$, където X е модифицираната първа половина (с дължина $2p + |yv| > 2p$). Новата дължина е $4p + |yv|$, а втората половина (последните $4p + |yv| - 2p - |yv|/2 = 2p + |yv|/2$ символа, ако $|yv|$ е четно) пресича границата.

По-просто: ако новата дума е $\omega'\omega'$, то по дължина двете половини имат по $2p + |yv|/2$ символа. Първите $2p$ символа на новата дума са модифицираната първа половина (от α), плюс още $|yv|/2$ символа. Тези $|yv|/2$ допълнителни символа идват от **втората половина** $a^p b^p$ — конкретно от началото му, т.е. от a -та.

Сега новата първа половина ω' съдържа в края си $|yv|/2$ a -та (от началото на старата втора половина). А новата втора половина ω' е суфикс от старата дума $a^p b^p a^p b^p$ — и завършва на b -та (от старата четвърта блокчета).

Така в новата първа половина буквата на края е a , а в новата втора половина — буквата на края е b . Те трябва да са еднакви, но не са. Противоречие.

Случай 2: uiv е изцяло във втората половина — симетричен на Случай 1, със същата идея, само че сега първата половина расте отзад. Структурата на двете нови половини отново не съвпада.

Случай 3: uiv пресича средата. Тогава $|uiv| \leq p$ означава, че uiv е изцяло в средните $2p$ символа: последните b -та от първата половина и първите a -та от втората половина. Помпването с $i = 2$ удължава именно тази зона.

Новата дума: $a^p b^{p+\Delta_1} a^{p+\Delta_2} b^p$, където $\Delta_1 + \Delta_2 = |yv| \geq 1$.

Нека $|yv|$ е четно (иначе новата дължина $4p + |yv|$ е нечетно и думата не е в L_c — край). Тогава всяка „нова половина“ има дължина $2p + |yv|/2$.

Новата първа половина обхваща a^p , всичките $b^{p+\Delta_1}$ (ако $p + \Delta_1 \leq p + |yv|/2$, т.е. $\Delta_1 \leq |yv|/2$), и евентуално някои a -та от втория a -блок. Новата втора половина пък започва някъде в средата на новия b -блок или a -блок и завършва с b^p .

Подробен анализ показва, че двете нови половини имат различна структура: в едната има два прехода между символи ($a \rightarrow b \rightarrow a$), в другата има друг брой преходи. По-конкретно, новата първа половина има вида „много a / много b / евентуално няколко a “.

", а новата втора половина има вида „евентуално няколко b / много a / много b " —
нямат еднакъв брой преходи. Противоречие.

Във всички случаи $xu^2uv^2w \notin L_c$. Затова L_c не е CFL. ■

Забележка. Доказателството за в) е по-техническо. Често за такива езици се ползва и **сечение с регулярен език**, което води до по-просто противоречие. Например: $L_c \cap \mathcal{L}(a^*b^*a^*b^*) = \{a^n b^m a^n b^m \mid n, m \geq 0\}$, и за този по-прост език с „двойна броеница“ се прилага лема за разрастването с по-чисто доказателство, аналогично на Случай 3 по-горе.

Обобщителна схема за решаване

Задача	Стратегия
Построй CFG за L	Разбий структурата: какво е вложено, какво е до? Използвай отделни нетерминали за всеки „слой“. Често помага да дефинираш помощни езици и да ги комбинираш чрез регулярните операции.
Докажи, че L е CFL	(1) Дай CFG и докажи коректността с индукция; (2) Конструирай стеков автомат; или (3) Изрази L чрез операции, относно които CFL са затворени (обединение, конкатенация, звезда, обръщане, сечение с регулярен, хомоморфизъм).
Докажи, че L не е CFL	(1) Лема за разрастването (контрапозиция) — главен инструмент; или (2) Покажи, че $L \cap R$ за някой регулярен R не е CFL; или (3) Сведи към известен не-CFL чрез операции, относно които CFL са затворени.

Кратка справочна таблица — свойства на езиковите класове

Свойство	CFL	Регулярни
Обединение	✓	✓
Конкатенация	✓	✓
Звезда на Клини	✓	✓
Сечение	✗	✓
Допълнение	✗	✓
Сечение с регулярен	✓	✓
Обръщане	✓	✓
Хомоморфизъм	✓	✓
Обратен хомоморфизъм	✓	✓

Незатвореността относно сечение и допълнение е едно от ключовите отличия между CFL и регулярните езици — и именно тя прави CFL „по-богат, но по-непокорен“ клас.